

AWS Certified Generative AI Developer — Professional

Domain 4, Task 4.1 — Cost optimization and resource efficiency · Revision notes

What This Task Is About

Domain 3 asked whether the system behaves correctly. Domain 4 asks: **how do you do the same work for less money?**

Much of this overlaps with Task 2.2 — cascading, caching, batching, provisioned throughput. The difference is the lens: **2.2 was "where do I run it", 4.1 is "what does it cost"**. Genuinely new here: token efficiency, semantic caching, prompt caching, price-to-performance measurement, and Cost Explorer attribution.

Bedrock cost optimisation — the order matters

1. Select the appropriate model
 2. Need customization? -> explore options in the correct order
 3. Prompt engineering and management
 4. Design efficient agents
 5. Select the correct consumption option

Model selection comes first; consumption option (on-demand vs provisioned) comes last. Choosing a cheaper model saves far more than tuning throughput ever will — pull the big levers first.

1. Token Efficiency Systems

In Bedrock you pay per token — **input tokens for what you send, output tokens for what the model generates**, and output is typically the more expensive of the two. Every extra word in a prompt costs money; every over-long answer costs more.

Token counting

```
count tokens BEFORE the API call
-> predict cost
-> confirm the prompt fits the context window
```

Track `InputTokenCount` and `OutputTokenCount` in CloudWatch **broken down by query type** — that reveals which query patterns consume the most, and those are the ones worth optimising.

Context window optimisation

Technique	Effect
Efficient chunking	Too large wastes tokens on noise; too small needs many chunks. Preserve semantic meaning while filling the window efficiently. (Task 1.5 chunked for retrieval quality — here it's for cost.)
Recursive summarization	Chunk → summarise → summarise the summaries. A 50-page document becomes 2 pages: ~25× fewer tokens . Implemented in Lambda.
Relevant information first	Put critical content at the start of the prompt, so it survives truncation — and because models attend more to the beginning (the "lost in the middle" effect).

Summarization itself costs a model call. It only pays off when the same document is used repeatedly — summarise once, reuse many times.

Prompt compression, response controls, context pruning

Technique	What to do
Prompt compression	Remove redundant words, shorten verbose instructions, trim unnecessary examples. Compress only until quality starts to drop — test it.
Response size control	Set a realistic <code>max_tokens</code> and instruct brevity in the prompt. Output tokens are the expensive ones.
Context pruning	Remove redundant or low-value content: RAG returned 10 chunks but only 3 are relevant; conversation history contains stale turns; boilerplate and headers. Score by relevance and drop below a threshold.

Same principle as noise reduction in Task 1.3 — **remove 30% of the noise and you remove 30% of the cost.**

Step Functions can automate testing of prompt structures to find the most token-efficient version.

2. Cost-Effective Model Selection

This is the biggest lever. Prompt optimisation might save 20%; choosing a cheaper model can save 80%.

Tiered model usage

Simple queries	-> small, inexpensive models
Medium complexity	-> mid-tier models
Complex queries	-> the most capable (and expensive) models, ONLY here

A **query analyzer** determines complexity and routes accordingly. The principle: *don't use expensive, highly capable models for basic tasks*.

Approach	Mechanism	Trade-off
Tiered routing	Assess complexity first, then call the right model	One call — cheaper
Cascading	Try the small model, escalate on low confidence	Two calls when it escalates — but based on real confidence

Systematic evaluation — cost against quality

1. Define quality metrics what counts as ACCEPTABLE quality for this use case?
2. Comparative testing Bedrock Model Evaluation across models, tracking cost alongside
3. Intelligent routing the CHEAPEST model that MEETS the quality threshold

Note the target: not the best model — the cheapest one that is good enough. Defining the quality threshold first is what makes that possible; without it, you will always pick the strongest model.

Price-to-performance ratio

Model A: 94% accuracy, \$0.015/query -> \$0.016 per accuracy point
Model B: 89% accuracy, \$0.003/query -> \$0.003 per accuracy point

Model B delivers roughly 5× the value per dollar — **but only if 89% clears your threshold**. Method: define performance metrics → compute cost per unit of performance → build a decision matrix.

Cost attribution — AWS Cost Explorer

inference-level cost allocation tags -> attribute model spend to business functions
--

"We're spending \$50,000 on GenAI" — **which team? which feature?** Without attribution you cannot optimise, because you don't know where the money goes. (Task 2.3's per-team usage plans throttled consumption; this measures it.)

Efficient inference patterns

WRONG: user opens a product page -> generate the description
RIGHT: nightly batch job -> pre-generate all descriptions

- Batch inference is roughly **50% cheaper** (Task 2.2)
- The same description isn't regenerated for every viewer
- **Rule: if real-time isn't required, batch it.**
- **Fallback to cheaper models** — in Task 1.2 this was for resilience; here it also serves cost, e.g. degrading during peak
- **Stopping conditions** prevent runaway consumption — the Task 2.1 safeguard, now motivated by cost: an agent stuck in a loop keeps billing

Cost optimisation is ongoing

regular monitoring	-> AWS cost management tools
performance analysis	-> evaluate against metrics
adjust selection criteria	-> refine as data changes
stay informed	-> new models and pricing changes

The last point is practical: a cheaper model launches, or pricing changes, and your framework is suddenly out of date.

3. High-Performance FM Systems

Batching — two different things

Type	Nature
Batch inference (Bedrock)	Offline job, ~50% cheaper — a cost feature
Request batching (runtime)	Multiple requests processed together on the GPU — a throughput feature

Capacity planning and utilisation

- GenAI load is hard to predict — token usage and latency both vary
- Load testing frameworks in Lambda** find the optimal configuration; encode it in **CloudFormation** for repeatable, right-sized deployments
- CloudWatch utilisation monitoring:** GPU at 20% means the instance is oversized; provisioned Model Units under-consumed means wasted spend, over-consumed means throttling

Auto scaling and concurrency

Setting	Cost implication
Reserved concurrency	A ceiling — prevents one function consuming account capacity
Provisioned concurrency	No cold starts, but you pay while idle — only worth it with predictable traffic

Provisioned throughput — the break-even

On-demand -> pay per token
Provisioned -> fixed commitment (Model Units)
Choose provisioned only when on-demand cost at your volume > the commitment

Otherwise you are paying for idle capacity. Monitor utilisation and adjust Model Units in both directions.

Queue-based architecture — the cost argument

Without a queue: you must provision for PEAK capacity, which sits idle the rest of the time
With SQS: AVERAGE capacity suffices; peaks wait in the queue

Task 2.2 used queues to avoid throttling; the cost benefit is **less infrastructure for the same workload**. The trade-off remains latency.

The three-way trade-off

want low latency -> provisioned capacity -> expensive
want low cost -> queue + batch -> higher latency
want throughput -> batching -> higher per-request latency

You cannot have all three. Pick two according to the requirement.

4. Intelligent Caching Systems

The simplest saving of all: don't call the model. A cached response costs almost nothing.

Semantic caching

"what is my balance?" "tell me my balance" "how much money is in my account?" Same answer -- but exact-match caching sees three different queries and makes three model calls.
query -> embedding -> find similar cached queries in a vector store (OpenSearch) -> similarity above threshold? -> return the CACHED response -> otherwise -> call the model -> cache the result

Threshold	Effect
Too low (~70%)	Returns answers to questions that weren't actually asked
Too high (~99%)	Effectively exact match — little benefit
85-95%	The usual sweet spot

Note this is the **third distinct use of embeddings** in this course: retrieval (1.5), verification (3.1), and now caching. Embedding the query does cost something, but far less than a model call.

Result fingerprinting and deterministic hashing

```
"What is my BALANCE?"    -> normalise -> "what is my balance"
"what is my balance "    -> normalise -> "what is my balance"
    -> same fingerprint -> same cache entry
```

- Cheaper than semantic caching (no embedding), but catches only **minor variations**, not paraphrasing
- **Deterministic hashing** matters in distributed systems — if the hash included a timestamp or any random element, every node would compute a different key and **the cache would never hit**

Prompt caching — a different thing entirely

```
system instructions  <- identical every call
few-shot examples    <- identical every call
retrieved context    <- sometimes identical
user question        <- varies
```

```
first call  -> the whole prompt is processed
later calls -> the cached prefix is reused;
              only the new portion is processed
```

- Cached tokens are heavily discounted (commonly around 90%), and **latency improves too**
- **The static portion must sit at the START of the prompt** — caching works on the prefix

```
CORRECT: [static instructions][examples][user query]
WRONG:   [user query][static instructions]
```

The notes also describe storing expensive prompt components yourself in **ElastiCache or DynamoDB**.

Edge caching and tiered architecture

- **CloudFront** caches frequently accessed content at the edge — good for FAQs and non-personalised generated content, useless for personalised responses
- **Tiered caching:** in-memory (fastest, smallest) → distributed ElastiCache (shared across instances) → edge CloudFront (closest to the user)

Invalidation and analytics

```
short TTL -> fresh data, fewer cache hits
long TTL  -> more savings, staler answers
event-based invalidation -> data changes, cache is cleared
```

Monitor hit rates in CloudWatch. A 5% hit rate means the cache is *costing* money (storage plus lookup) rather than saving it. Measure before assuming.

The four caching types

Type	Caches	Use when
Semantic	Full response, matched by similarity	Users ask the same thing in different words
Fingerprinting	Response, matched by normalised query	Minor variations — case, spacing, punctuation
Prompt caching	The static prefix of the prompt	Large system prompts and few-shot examples
Edge (CloudFront)	Response, geographically distributed	Common, non-personalised content

Service limits, pricing and feature availability change. Verify specific numbers against current AWS documentation before the exam.